StenoWav

User Guide

Version 0.1.0

StenoWav Project

March 2026

Contents

1 What Is StenoWav? 3
- 1.1 Key Features . . . 3

2 Quick Start 3
- 2.1 Hide a Message . . . 3
- 2.2 Recover the Message . . . 3

3 How It Works (The Simple Version) 4
- 3.1 The Basic Idea . . . 4
- 3.2 Step by Step . . . 4
- 3.3 Why Is It Secure? . . . 4

4 Configuration 4
- 4.1 Choosing a Domain . . . 5
- 4.2 Builder Methods . . . 5
- 4.3 All Parameters . . . 5
- 4.4 Direct Field Access . . . 5

5 API Reference 6
- 5.1 File-Based API . . . 6
- 5.2 In-Memory API . . . 6
- 5.3 Error Handling . . . 6

6 Architecture Overview 7

7 Understanding QIM (Simplified) 8

8 Practical Guidelines 8
- 8.1 Audio Requirements . . . 8
- 8.2 How Much Data Can I Hide? . . . 8
- 8.3 Choosing Delta . . . 9
- 8.4 Key Management . . . 9

9 Examples 9
- 9.1 Hide a Text Message . . . 9
- 9.2 Hide a Binary File . . . 9
- 9.3 Frequency Domain with Custom Parameters . . . 10
- 9.4 Working with Samples Directly . . . 10

10 Troubleshooting **11**

11 Future Roadmap **11**

12 Glossary **12**

1 What Is StenoWav?

StenoWav hides secret data inside ordinary audio files. You give it a WAV file, a message (any bytes), and a secret key. It gives you back a WAV file that sounds exactly the same — but carries your message inside it. Anyone with the same key can extract the message; everyone else just hears music.

This is called **audio steganography**: hiding information inside audio in a way that is invisible to the ear and difficult to detect without the key.

1.1 Key Features

- **Two embedding modes**: hide data directly in audio samples (time domain) or in the frequency spectrum.
- **Smart placement**: the framework analyses the audio and only hides data in loud sections, where small changes are masked by the music itself.
- **Encryption**: each packet of hidden data is encrypted with a unique cipher derived from the audio itself.
- **Self-contained**: the watermarked file contains everything needed for extraction — just provide the key.
- **Optional noise masking**: an extra "dither" pass makes statistical detection even harder.
- **No Python**: built entirely in Rust and C for performance and safety.

2 Quick Start

2.1 Hide a Message

```
use stenowav::steno::{Steno, StenoConfig};

// Your secret key (32 bytes --- like a password)
let key = [42u8; 32];

// Default settings work out of the box
let config = StenoConfig::default();

// Hide "Hello, world!" inside a WAV file
Steno::encode(
    "original.wav",       // input audio
    "watermarked.wav",    // output audio (sounds the same!)
    b"Hello, world!",     // your secret message
    &key,
    &config,
).unwrap();
```

2.2 Recover the Message

```
// Same key and config as encoding
let recovered = Steno::decode(
    "watermarked.wav",
    &key,
    &config,
```

```
).unwrap();

assert_eq!(recovered, b"Hello, world!");
```

That's it. Two function calls — one to hide, one to recover.

3 How It Works (The Simple Version)

3.1 The Basic Idea

Audio is stored as thousands of numbers per second (called **samples**). Each sample represents how loud the sound is at that instant. For 16-bit audio, each sample is an integer from -32768 to $+32767$.

StenoWav makes tiny changes to some of these numbers. The changes are so small that your ears can't tell the difference, but a computer with the right key can read them back.

3.2 Step by Step

1. **Read the audio**: Load the WAV file into memory.
2. **Find good hiding spots**: Analyse the audio to find the loud sections. Loud music masks small changes better than silence does. Quiet sections are left untouched.
3. **Pick random positions**: Using the secret key as a seed, a random number generator picks exactly which samples to modify. Without the key, an attacker doesn't know which samples to look at.
4. **Create a cipher**: For each chunk of data, 8 "mask" samples are modified to encode a 16-bit encryption key. This key is then used to encrypt the actual data before embedding it.
5. **Embed the data**: The encrypted data bits are hidden in 16 more samples using a technique called QIM, which rounds each sample to the nearest value on a mathematical grid that encodes the desired bit.
6. **Add noise (optional)**: A tiny amount of random noise is added to all the *other* samples, so the modified ones don't stand out statistically.
7. **Write the output**: Save the modified audio as a new WAV file. It sounds identical to the original.

3.3 Why Is It Secure?

Three things protect your data:

- **Position**: Without the key, you don't know *which* samples carry data. There could be millions of samples.
- **Encryption**: Even if you found the right samples, the data is XOR-encrypted with a cipher derived from the audio itself.
- **Stealth**: With dither enabled, the entire audio has micro-noise, so modified and unmodified samples look the same under statistical analysis.

4 Configuration

StenoWav works out of the box with defaults, but you can tune every parameter.

4.1 Choosing a Domain

```
// Time domain (default) --- modifies audio samples directly
let config = StenoConfig::time();

// Frequency domain --- modifies the frequency spectrum
let config = StenoConfig::frequency();
```

Which domain should I use?

Time domain is simpler and faster. Good for most use cases.
Frequency domain spreads changes across many time-domain samples (via the inverse FFT), which can be harder to detect but uses more computational resources.
Both modes are well-tested and produce inaudible results on typical music.

4.2 Builder Methods

Chain methods to customise:

```
let config = StenoConfig::time()
    .with_dither()            // enable noise masking
    .with_time_delta(0.01)    // larger step = more robust
    .with_channel(1);         // embed in right channel
```

4.3 All Parameters

Parameter	Default	What it does
`domain`	Time	Where to hide the data
`time_delta`	0.005	Size of the rounding grid (time domain). Larger = more robust to noise but slightly more audible
`freq_delta`	0.05	Same but for frequency domain
`rms_high`	0.10	Loudness threshold for "high power". Sections above this get embedded in
`rms_low`	0.01	Sections below this are skipped entirely
`rms_segment_size`	4096	How many samples per loudness measurement
`freq_window_size`	1024	FFT window size (must be a power of 2)
`freq_min_bin`	10	Skip very low frequencies
`freq_max_bin`	400	Skip very high frequencies
`dither_enabled`	false	Add noise to non-data samples
`dither_intensity`	~6.1e-5	How much noise (~2 least significant bits)
`channel`	0	Audio channel (0 = left, 1 = right)

4.4 Direct Field Access

All fields are public, so you can set them directly too:

```
let mut config = StenoConfig::default();
config.rms_high = 0.15;
config.rms_low = 0.02;
config.dither_enabled = true;
```

5 API Reference

5.1 File-Based API

These are the main functions most users need:

Steno::encode

```
Steno::encode(
    input_path,     // path to source WAV (16-bit PCM)
    output_path,    // path to write watermarked WAV
    data,           // &[u8] --- any bytes you want to hide
    key,            // &[u8; 32] --- your secret key
    config,         // &StenoConfig
) -> Result<(), StenoError>
```

Steno::decode

```
Steno::decode(
    watermarked_path,  // path to watermarked WAV
    key,               // &[u8; 32] --- same key as encoding
    config,            // &StenoConfig --- same config as encoding
) -> Result<Vec<u8>, StenoError>
```

5.2 In-Memory API

For when you already have audio samples loaded:

Steno::encode_samples / decode_samples

```
// Modify samples in-place
Steno::encode_samples(
    samples,  // &mut [f64] --- audio in [-1.0, 1.0]
    data, key, config,
) -> Result<(), StenoError>

// Extract from samples
Steno::decode_samples(
    samples,  // &[f64]
    key, config,
) -> Result<Vec<u8>, StenoError>
```

5.3 Error Handling

All functions return `Result` with `StenoError`:

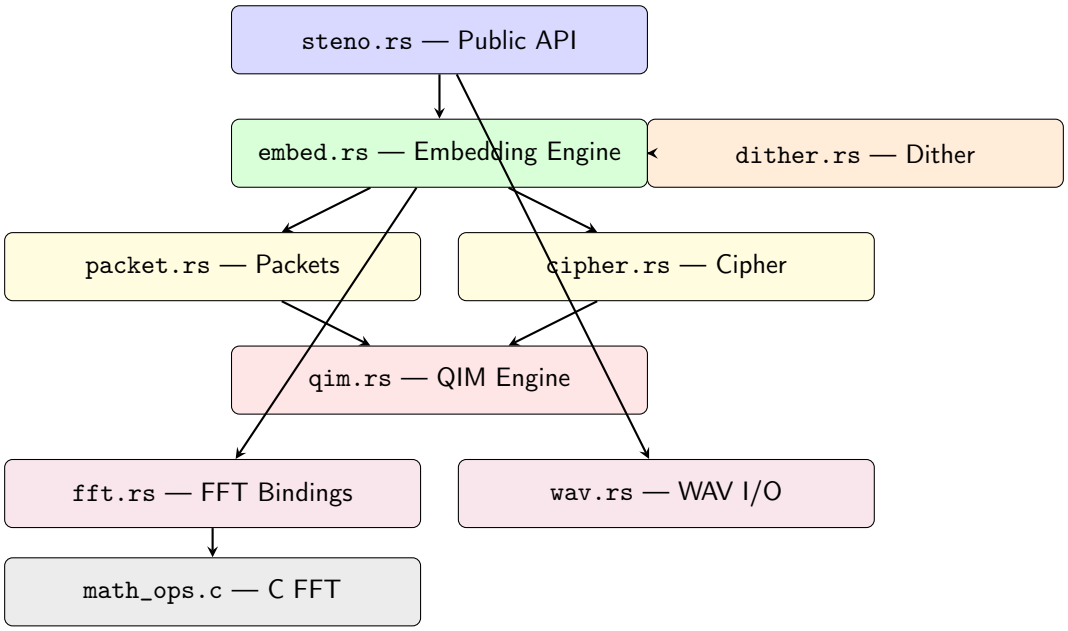
- `StenoError::Wav` — file not found, not a valid WAV, etc.
- `StenoError::Embed` — audio too short/quiet, wrong key, etc.
- `StenoError::Config` — invalid parameters (negative delta, non-power-of-2 window, etc.)

Wrong key behaviour

If you try to decode with the wrong key, you get an error (not garbage data). The framework checks for a magic signature in the header — a wrong key produces an invalid signature and returns `StenoError::Embed(InvalidConfig(...))`.

6 Architecture Overview

StenoWav is built in layers, each handling one concern:



`steno.rs`
: The front door. Handles file I/O, config validation, channel selection. Everything goes through here.

`embed.rs`
: The brain. Decides where to embed (RMS analysis), manages headers, dispatches to time or frequency path.

`packet.rs`
: The workhorse. Selects random sample positions, encodes/decodes individual packets.

`cipher.rs`
: The locksmith. Generates and extracts per-packet encryption keys from mask samples.

`qim.rs`
: The math. The core quantisation algorithm that turns a sample and a bit into a modified sample.

`dither.rs`
: The camouflage. Adds inaudible noise to non-payload samples. Also provides quality metrics (SNR, etc.).

`fft.rs`
: The bridge. Safe Rust wrappers around the C FFT library.

`wav.rs`
: The translator. Reads and writes 16-bit PCM WAV files to/from `f64` sample arrays.

`math_ops.c`
: The engine room. Radix-2 Cooley-Tukey FFT in pure C.

7 Understanding QIM (Simplified)

Imagine a number line with marks every 0.01 units:

$$\ldots, -0.02, -0.01,\ 0.00,\ 0.01,\ 0.02,\ 0.03, \ldots$$

Now colour the marks alternately: **blue** (even index) and **red** (odd index):

$$\ldots, -0.02, -0.01, 0.00, 0.01, 0.02, 0.03, \ldots$$

To hide a **0**, round the sample to the nearest blue mark. To hide a **1**, round it to the nearest red mark.

To read the bit back, round the sample and check its colour.

That's QIM. The spacing (0.01 in this example) is called **delta** (Δ). Bigger delta = the mark is harder to accidentally knock off (more robust), but the rounding changes are bigger (more audible).

8 Practical Guidelines

8.1 Audio Requirements

- **Format**: 16-bit PCM WAV (mono or stereo, any sample rate).
- **Length**: Longer audio = more room for data. At least a few seconds of music for short messages.
- **Content**: Music with dynamic range works best. Pure silence or very quiet recordings have fewer eligible hiding spots.

8.2 How Much Data Can I Hide?

Each "packet" carries 2 bytes and uses about 24 sample positions. The framework also needs 7 packets for its internal header.

Audio length (44.1 kHz mono)	Approximate capacity
10 seconds (∼441K samples)	∼700 bytes
1 minute (∼2.6M samples)	∼4 KB
3 minutes (∼7.9M samples)	∼14 KB

Capacity depends on content

These numbers assume roughly half the audio is "loud enough" for embedding. Quiet recordings or recordings with lots of silence will have lower capacity.

8.3 Choosing Delta

Delta	Robustness	Audibility
0.003	Lower (fragile)	Very imperceptible
0.005	Good (default)	Imperceptible
0.01	High	Still imperceptible
0.02	Very high	Borderline (quiet passages)

The default (0.005) is a safe all-rounder. Only increase it if you need extra robustness against noise or processing.

8.4 Key Management

- The key is 32 bytes (256 bits). Use a cryptographically secure random generator to create it.
- Both encoder and decoder must use **exactly the same key and config**. Any mismatch will fail.
- Store keys securely — anyone with the key can extract the hidden data.

9 Examples

9.1 Hide a Text Message

```
use stenowav::steno::{Steno, StenoConfig};

fn main() {
    let key: [u8; 32] = [
        0x1A, 0x2B, 0x3C, 0x4D, 0x5E, 0x6F, 0x70, 0x81,
        0x92, 0xA3, 0xB4, 0xC5, 0xD6, 0xE7, 0xF8, 0x09,
        0x10, 0x21, 0x32, 0x43, 0x54, 0x65, 0x76, 0x87,
        0x98, 0xA9, 0xBA, 0xCB, 0xDC, 0xED, 0xFE, 0x0F,
    ];

    let config = StenoConfig::time().with_dither();

    Steno::encode(
        "my_song.wav",
        "my_song_secret.wav",
        b"Meet␣me␣at␣the␣bridge␣at␣midnight",
        &key,
        &config,
    ).expect("encoding␣failed");

    println!("Message␣hidden␣successfully!");
}
```

9.2 Hide a Binary File

```
use stenowav::steno::{Steno, StenoConfig};
use std::fs;

fn main() {
```

```
    let key = [42u8; 32];
    let config = StenoConfig::time();

    // Read any file as bytes
    let secret_file = fs::read("secret_document.pdf")
        .expect("could not read file");

    Steno::encode(
        "music.wav",
        "music_with_pdf.wav",
        &secret_file,
        &key,
        &config,
    ).expect("encoding failed");

    // Later: recover it
    let recovered = Steno::decode(
        "music_with_pdf.wav",
        &key,
        &config,
    ).expect("decoding failed");

    fs::write("recovered_document.pdf", &recovered)
        .expect("could not write file");
}
```

9.3 Frequency Domain with Custom Parameters

```
use stenowav::steno::{Steno, StenoConfig};

fn main() {
    let key = [7u8; 32];

    let config = StenoConfig::frequency()
        .with_freq_delta(0.08)
        .with_dither();

    Steno::encode(
        "podcast.wav",
        "podcast_watermarked.wav",
        b"Copyright 2026 - My Podcast",
        &key,
        &config,
    ).expect("encoding failed");
}
```

9.4 Working with Samples Directly

```
use stenowav::steno::{Steno, StenoConfig};

fn main() {
    let key = [0u8; 32];
    let config = StenoConfig::time();

    // Generate a synthetic signal (e.g., 440 Hz tone)
    let mut samples: Vec<f64> = (0..200_000)
```

```
        .map(|i| {
            let t = i as f64 / 44100.0;
            0.8 * (2.0 * std::f64::consts::PI * 440.0 * t).sin()
        })
        .collect();

    Steno::encode_samples(
        &mut samples,
        b"Hidden in a sine wave!",
        &key,
        &config,
    ).unwrap();

    let recovered = Steno::decode_samples(
        &samples,
        &key,
        &config,
    ).unwrap();

    assert_eq!(recovered, b"Hidden in a sine wave!");
}
```

10 Troubleshooting

"header error: invalid magic"
The key, config, or domain doesn't match what was used for encoding. Make sure both sides use the exact same `key` and `StenoConfig`.

"insufficient capacity"
The audio is too short or too quiet. Try:

- Using longer audio
- Lowering `rms_low` (include quieter sections)
- Hiding less data

"channel X does not exist"
You selected a channel number that doesn't exist in the file. Mono files only have channel 0.

"invalid WAV format"
The file is not a 16-bit PCM WAV. Convert it first (e.g., with `ffmpeg -i input.mp3 -acodec pcm_s16le output.wav`).

11 Future Roadmap

Planned features for future versions:

1. **Variable packet sizes**: Currently fixed at 9 samples per packet; future versions may support larger packets for higher data density.
2. **Higher bit density**: Embed more bits per sample using higher-order QIM. Analysis via AWGN simulation sweeps will determine optimal parameters.
3. **Compression robustness**: Harden against MP3, AAC, and Opus transcoding.

4. **Error correction**: Add Reed-Solomon or similar codes so data survives even if some samples are corrupted.
5. **Streaming mode**: Real-time embedding and extraction for live audio applications.
6. **Multi-channel embedding**: Spread data across stereo channels for increased capacity.
7. **Python analysis tools**: Parameter optimisation scripts using AWGN noise sweeps (separate from the core Rust+C framework).

12 Glossary

CSPRNG
Cryptographically Secure Pseudo-Random Number Generator. A random number generator that is unpredictable without the seed. StenoWav uses ChaCha20.

Delta (Δ)
The QIM step size. Controls the spacing of the quantisation grid. Larger delta = more robust but more audible.

DFT / FFT
Discrete Fourier Transform / Fast Fourier Transform. Converts a signal from the time domain (amplitude over time) to the frequency domain (energy at each frequency).

Dither
Small random noise added to non-payload samples to mask the statistical signature of QIM embedding.

Magnitude
The "strength" of a frequency component, computed as $\sqrt{\text{Re}^2 + \text{Im}^2}$. Always non-negative.

Mask samples
8 samples per packet whose QIM residues form a per-packet cipher key.

Packet
The fundamental unit of embedding: 8 mask samples + 1 data region (16 consecutive samples) = 24 samples total, carrying 2 bytes.

PCM
Pulse Code Modulation. The standard way digital audio is stored: each sample is a number representing amplitude.

QIM
Quantisation Index Modulation. A technique that encodes information by rounding samples to specific values on a grid.

RMS
Root Mean Square. A measure of the "average loudness" of a signal segment.

Seed / Key
A 32-byte value that controls all random choices (sample positions, cipher generation, dither noise). Shared secret between encoder and decoder.

Steganography
The practice of hiding information inside other, innocent-looking data (in this case, audio).

WAV
A common uncompressed audio file format. StenoWav works with 16-bit PCM WAV files.